基于道路实用性的校园物流集散模型

摘要

本文主要研究配送机器人完成还书、借书任务的最短时间。本文根据不同影响因素建立模型以寻求完成任务的最短路径及时间。

根据本文建立的模型，本文得出只完成还书任务的总时间为5.09小时，同时完成还书和借书任务的总时间为5.13小时。

首先，本文解决只考虑还书任务的情况。本文将该问题分为两个部分：先计算完成任务的最短路线，再将最短路径分配给两个机器人。构建路径所建立的模型采用 TSP 问题中的贪心思想与决策树的结合：每当选择道路和节点时，总是选择当前最优的和期望能达到整体最优的路径，并预测当前决策对未来所产生的影响。决策的制定将基于文中所定义的“道路实用性”。

有关路径的计算主要考虑以下因素：每栋楼的还书量、每栋楼到图书馆的最短距离、机器人的速度、机器人的起始点以及机器人的最大装载量。本文使用邻接表建图，以每栋楼为节点（需求量为点权），以道路为边（长度为边权）；使用实用性表示有多少个非零需求节点的最短回程路径经过该边。

对每个节点，模型优先考虑与之相连的边的实用性。如果该点只有一条实用性大于零的边与之相连，就说明这个点的实用性小，访问率低，在本文中也被称之为“气口”。每当走到一个点之后，会依次判断其周围的点：是否存在唯一“气口”、能否装完该点全部还书以及回程距离长短。本文使用计算机对模型进行求解与计算，得到最小总路程为94.4公里。在此基础上，考虑到部分路径在全局剩余量较少的情况下无法最优装载量，本文又针对迭代末期的特点建立了基于装载量的点群分类算法，合并了相关路径，最终得到总路程为91.6公里。

由于每条路径不会互相影响，本文根据 AB 机器人的速度按比例分配 AB 机器人各自的总路程，最终得到还书任务总时间为5.09小时。

此外，本文解决了同时存在借书与还书的情况。对于同时借还的情形，本文沿用了仅还书情况下的模型，并将问题分为最短路径和最短时间两部分分别计算。由于机器人的装载量有限，机器人每次将只携带一条路径所需要的总借书量。经过计算，本文发现只有一条路径上的一个节点的需求无法被机器人在取还书时满足，需要在另外的一条路径中前往该点。最终，本文使用计算机对借还书的路径进行了求解，得出最小总路程为92.3公里；还书总时间为5.13小时。

关键词：物流模型 图论 最短路模型 仿真 旅行商问题

目录

摘要 1
1. 引言 3
2. 问题重述与文献综述 3
3. 基本假设 4
4. 符号约定与名词定义 4
4.1. 符号约定 4
4.2. 名词定义 6
4.2.1. 道路实用性 6
4.2.2. 气口 6
4.2.3. 死胡同 6
5. 影响因素分析 7
5.1. 路径相关因素 7
5.1.1. 楼宇还书量 7
5.1.2. 楼宇借书量 7
5.1.3. 最短回程距离 7
5.1.4. 气口数量 7
5.2. 机器人自身差异因素 7
5.2.1. 速度 8
5.2.2. 充电桩 8
5.2.3. 最大装载量 8
6. 还书任务 8
6.1. 基本数据输入 8
6.2. 初始位置选取 9
6.3. 路径选择策略 9
6.4. 路径求解与优化 9
6.5. 路程分配 13
6.6. 灵敏度分析 14
7. 借还书任务 14
7.1. 额外影响 14
7.2. 模型异同对比 15
8. 结论和进一步讨论 15
8.1. 整体评价 15
8.2. 优势和劣势 15
8.3. 未来研究方向 15
参考文献 17
附录 18
/REQUIREMENTS.TXT （库依赖） 18
/DEFINES.PY （常量定义） 18
/GRAPH.PY （图模块 + 主程序） 18
/DATA/GRAPH.TXT （图数据） 29

1. 引言

随着人们生活水平的不断提升，物流配送逐渐成为了现代生活圈中不可或缺的一部分。不少高校，也在尝试利用智慧物流系统来处理校园内庞大的图书借还需求。其中，如何设计机器人的路线来更有效率的归还和借阅所有书籍，成为了相关领域的研究热点。

2. 问题重述与文献综述



图 1. 校园道路与楼宇示意图

以上图所示校园为例。在借还书的过程中，机器人可执行的方法非常多，不同的方法造成的行驶总时间也不同，路程、数量、AB 机器人都有可能造成不确定因素。例如，A 机器人先向西走、再向北到图书馆，或先向西走到沉璧楼再向北、向西走，其路程长度都不一样。同时，AB 机器人运行速度差异也会影响到整体的效率。只有设计出合理、高效率的行驶路线，才能让机器人以尽量短的时间完成借书、还书的任务。

此类问题最原始的版本是 Dantzig 等人在 1959 年提出的旅行商问题（TSP）[1]。该问题希望寻找一条保证通过所有需求点的最小成本路径。当需求点数量增多后，TSP 的求解开始变

的复杂，因此大多数旅行商问题的求解都需要借助计算机的帮助[2]。

在经典旅行商问题中，旅行商背包拥有无限大的容量，且地图中的每条路径只能拜访一次。1959 年，VRP 模型的提出对 TSP 模型进行了拓展，解决每条路径的访问次数限制[3]。之后，CVRP 又对 VRP 中的车辆容载量进行了约束。如今，CVRP 模型已经十分成熟，其中遗传算法[6]、模拟退火算法[8]与蚁群算法[7]是常用的求解手段。不过，CVRP 的缺点也很明显：它在大规模数据上的求解不够稳定，且时间复杂度较高[5]。

本文参考旅行商问题中的贪心法思想，建立了决策树，并引入“道路实用性”指导其迭代。在此基础之上，增加考虑还书和借书的情况，得到高效的解决方法。

3. 基本假设

假设 1：借书量远小于还书量。根据题目中所给的数据，每栋楼宇所需求的借书量普遍低于还书量，因此在还书和借书同时进行的过程中，借书对还书过程的影响微乎其微，应优先保证还书过程的完整性。

假设 2：机器人只在返回途中取书。由于机器人容量有限，对于每条从图书馆出发再次回到图书馆的路径，机器人只会在从初始位置到图书馆的路线上取书，不会在前往初始位置的路上取书，否则前半程所选取的策略会影响返回时的策略，反之亦然，导致结果在迭代时无法收敛。

假设 3：机器人在取书过程中不会选择跳过某个楼宇。若允许机器人跳过楼宇，则在t时刻的机器人需要判定跳过$t+1$时刻后对$t+2$时刻的影响，而判断$t+2$则又会用到$t+3$时刻的结果，以此类推，最终导致决策树深度过高严重影响求解性能。通过禁止机器人的“跳过”功能，其容量会自动将决策树深度限制在一定范围内，保证迭代速度。

4. 符号约定与名词定义

4.1. 符号约定

表 1. 符号约定

符号	意义
r_i	第i点需求的还书量
b_i	第i点需求的借书量
(r_i,b_i)	第i点属性的表示数对（参考r_i、b_i定义）

R	机器人的剩余容量
C	机器人的总容量
$e_{i,j}$	第i点与第j点之间的直连道路长度，与$e_{j,i}$相等
$d_{i,j}$	第i点与第j点之间的最短路径长度，与$d_{j,i}$相等
$u_{i,j}$	第i点与第j点之间的直连道路实用性，与$u_{j,i}$相等
v_i	第i点的累计道路实用性
w_i	第i点所相连的气口数量（*气口*定义见 4.2.2）

最短路径的计算在图论中已有成熟的方案。若无特殊说明，本文均使用邻接表存图、Dijkstra 算法求最短路。

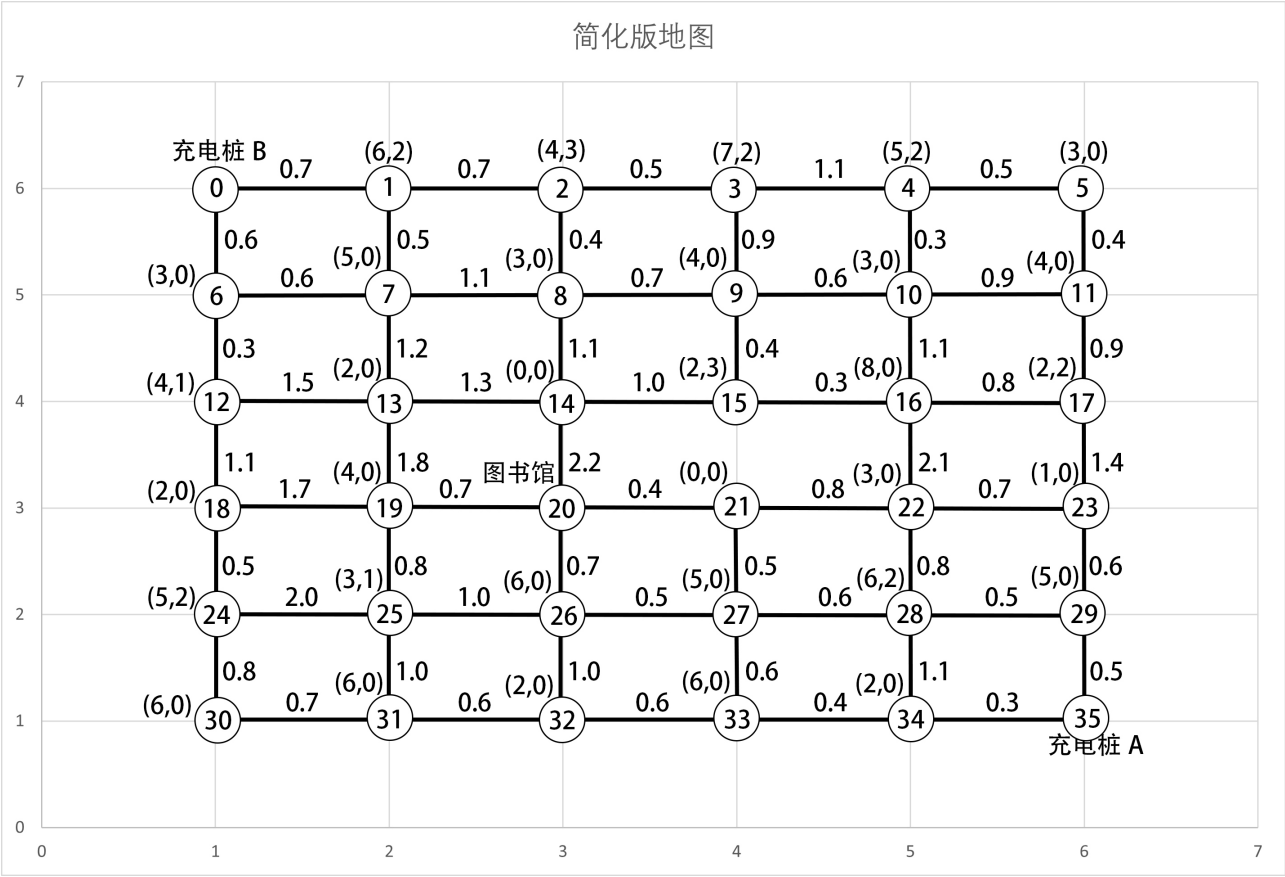


图 2. 简化版地图

如上图所示，根据每栋楼的还书量和借书量给每栋楼赋值(r_i, b_i)，并将每栋楼设为一个个节点，道路距离则是边，将这些点相连，建立相关图论模型，可以得到一个网格。网格中包

含每个节点（每栋楼）的还书量与借书量以及每条道路的长度。

4.2. 名词定义

4.2.1. 道路实用性

道路实用性$u_{i,j}$是指最短回程路线经过这条道路的需求非零楼宇（节点）个数，当越多最短回程路线经过该条道路时，该条道路的实用性就越大。我们可以将道路实用性通过颜色深浅程度进行表示：我们将每个节点的最短回程路径所经过的道路刷上颜色，颜色越深代表经过次数越多，反之亦然。由此，道路越浅，道路实用性越低；道路越深，道路实用性越高。

4.2.2. 气口

“气口”一词来源于围棋。这里气口是指机器人在某个点准备进行移动决策时，与之相连的道路中只有一条被“染”过颜色道路，将该点称之为“气口”。该点是气口，则表明单独前往该点的收益高，因为其访问率低。附近气口数量越多的点，表明前往该点与附近点付出的路程代价也越大。

为了计算气口数量，我们需要先定义“累计道路实用性”v_i，即与点i相连所有道路的实用性之和：

$$v_i = \sum_j u_{i,j}$$

则气口数量w_i定义为与点i直连的所有点的累计道路实用性为1的个数：

$$w_i = \sum_j \begin{cases} 1\ (v_j = 1) \\ 0\ (v_j \neq 0) \end{cases}$$

联立两式可得：

$$w_i = \sum_j \begin{cases} 1\ (\sum_k u_{j,k} = 1) \\ 0\ (\sum_k u_{j,k} \neq 1) \end{cases}$$

4.2.3. 死胡同

当机器人所处位置周围气口数为零时，称机器人所在的点为死胡同。机器人在非必要的情况下会更倾向于向不是死胡同的节点走，也就是前往收益更高的点。

5. 影响因素分析

本文认为影响整体效率的因素大体可分为两类：路径相关因素与机器人自身差异因素。

5.1. 路径相关因素

5.1.1. 楼宇还书量

在本模型中，每栋楼有一定的还书量，这预示着机器人是否还要经过该楼。如果某楼宇还书量为0，那么在取书过程中将不选择经过这栋楼，而选择其他路线取书。由于在机器人去往一个楼的过程中不取书（基本假设），并选择沿最短路径行走，所以只有取书路线是规划路径时所关注的内容。当一栋楼相邻的多栋楼的气口数量相同且均不为0和1的情况下，机器人会偏向走能拿完邻点楼所有书的路径，这样可以减少机器人在取书过程中再次经过这个点而造成路径重复、时间增加的情况。

5.1.2. 楼宇借书量

借书量只在第二个问题中产生影响。同样的，在模型中，每栋楼的借书量和还书量只要不都为0，则机器人必须要经过该楼。在机器人从图书馆出发再回到图书馆的过程中，机器人可以同时进行拿书和还书的任务。当然，这还会和单个机器人承载量有关，相关内容会在机器人承载量因素部分详细说明。

5.1.3. 最短回程距离

一栋楼的最短回程距离是指机器人从一栋楼回到图书馆的最短路径距离。它会影响并决定机器人更应该从哪个点开始取书并返程。同时，最短回程距离也会影响路径选择。由于重复走离图书馆较远回程距离的点会增加整体时间，机器人在取完一栋楼书后，会更倾向去其临近的楼中距离图书馆更远回程距离的点，这样可以缩短整体路线的时间。

5.1.4. 气口数量

机器人总会向气口多的地方前进，因为这样可以让机器人以最小的代价将这些收益高的楼宇所需求归还的书拾取完毕。

5.2. 机器人自身差异因素

除机器人需要走的所有最佳路径之外，两个机器人不同的特性也需要在模型中给予考虑。

5.2.1. 速度

由于 A 机器人和 B 机器人的速度不一样，AB 机器人的效率不同，两个机器人分别走的总路径长度不应该相同。为了使完成任务时间最短，应根据机器人的速度比例找出最适合两个机器人分别走的路线比例，从而使两个机器人尽可能同时完成任务。

5.2.2. 充电桩

A 和 B 机器人的初始点和结束点也是不同的，所以这些点到图书馆的距离也不相同。由于在仅还书的模型中，机器人在最开始从充电桩走向图书馆可以完成还书任务，最后从图书馆回到充电桩时可以完成借书任务，所以 AB 两个机器人的充电桩位置对从图书馆出发、取书、回到图书馆的时间以及一些楼的借书、还书量均会产生不同影响。

5.2.3. 最大装载量

每个机器人的最大装载量是固定的10本。在还书任务中，装载量决定了能不能拿完该点所有需要还的书。倘若无法拿完，机器人需要再前往该点将该点的书拿完;同时也有可能在另外的路径中，在没有达到最大装载量的情况下，机器人可以将未拿完的书顺带拿完。这些都会影响路线规划和总路线时间。在第二个同时还书和借书的任务中，最大装载量会影响机器人能否一次同时将一栋楼的借书还书任务完成，如果无法完成，机器人还需要再次经过该楼执行任务，从而影响行驶路线和时间。

由于机器人的速度和装载量都是固定的值，所以在模型中会最后考虑到这些因素。但是 AB 从不同的起始点去图书馆可能采取不同的路径，导致楼宇还书量改变，因此需要相对优先考虑这些因素。

6. 还书任务

本文首先只考虑需要还书的情况。整个还书任务可以被分成三个部分：1）机器人从充电桩出发，途径目标建筑物取书，再回到图书馆；2）机器人在图书馆放完书，再出发取书并返回图书馆；3）机器人完成所有还书任务后，返回充电桩起始点。对于从充电桩到图书管的路线和从图书馆再回到图书馆的路线，本文将建立还书模型进行优化。对于最后一部分，只需计算机器人从图书馆到充电站的最短路径即可。

6.1. 基本数据输入

本模型中，不同楼的还书量、最短回程距离、相邻气口数量、机器人的剩余装载量以及对之前路径的规划都会对当前的路径规划策略产生影响。此外，机器人的各自起始点与行驶

速度也会影响 AB 完成还书任务的总时间。在此，本文先讨论了只有单个机器人时的路径规划模型。

6.2. 初始位置选取

初始位置的选择会考虑该点是否为气口、周围的气口数量以及该点到图书馆到最短回程距离。首先，模型会挑选出是气口的节点，然后判断当这些气口被选取时、去掉该点最小回程路径（也就是该条道路的颜色）的时候，周围拥有气口数量最多的节点。如果出现一样多的情况，模型会选择最短回程距离最长的节点。

6.3. 路径选择策略

当确定初始位置之后，我们将刷新全图的道路实用性颜色，即使用去掉该初始位置的最短回程路线后的图，并判断该点周围的气口情况。同时，在判断路径方向时，每走到一个节点，机器人若能将该点的全部书都拿完，该点的最短回程路线就会被删除，并刷新全图道路实用性的颜色；当出现无法装载该点所有书的时候，该点的最短回程路线不会被删除，也不会刷新全图的道路实用性颜色。

如果该初始位置周围只有一个气口，那么机器人将走到这个节点。如果初始位置周围有多个气口，则分三种情况讨论：

1) 如果机器人走到所有相邻气口取书，都能保证所有书都被拿完，那么机器人会倾向于选择该这些气口中图书馆回程距离最长的点，从而保证机器人不会重复走更远的路线。
2) 如果机器人走到所有相邻气口取书，都无法拿完所有的书的话，机器人会选择距离图书馆更近的点。因为此时这些点的书都无法保证拿完，就说明机器人在此次取满后，还需要再走到这些点一次，所以此时应选择回程路径最短的点以尽快回到图书馆准备下一次出发。
3) 如果机器人走到所有相邻气口取书，存在部分气口的书无法拿完、部分气口的书可以拿完，则此时会选择能拿完书的节点走。如果有多个能拿完书的节点，将使用第二种情况的判断方法。

当机器人行进到一个节点的时候，碰到周围有多个气口的情况，都将按照相同的判断方式进行判断并选择路径。

除了碰到多个气口的情况，机器人还可能会走到“死胡同”。当机器人走到“死胡同”时，采用的方法与多气口情况的方法相同。

6.4. 路径求解与优化

使用计算机，我们根据上述决策策略进行了仿真计算。仿真总共经过14次迭代后结束。我们对求解过程中每一次迭代时的道路实用性情况进行了作图，形成了以下这种类似“剪枝”的收敛图像：

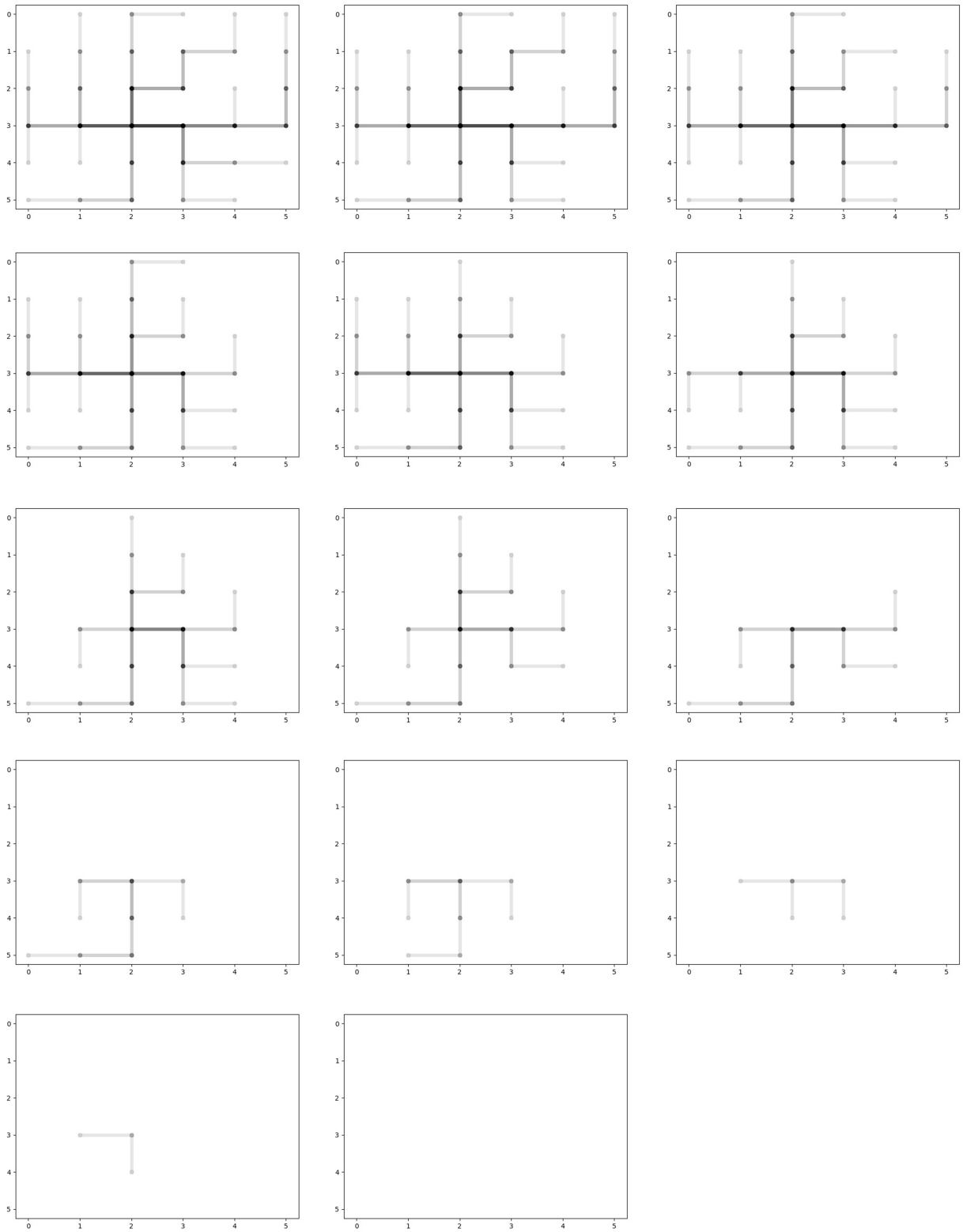


图 3. 路径求解收敛图

通过将各段路程相加求和，我们得出最终的取书总路程为94.4km。这些路径的汇总情况如下表所示：

表 2. 取书路径汇总

名称	路径	长度(km)	取书量(本)
充电桩 A 出发	[35, 29, 28, 28, 27, 21, 20]	2.5	10
充电桩 B 出发	[0, 1, 7, 7, 13, 19, 20]	4.9	10
初始位置4	[20, 14, 15, 9, 10, 4, 5, 11, 17, 23, 22, 21, 20]	9.6	10
初始位置10	[20, 14, 15, 9, 10, 11, 17, 23, 22, 21, 20]	9.3	10
初始位置3	[20, 14, 8, 2, 3, 9, 15, 14, 20]	8.7	10
初始位置6	[20, 19, 18, 12, 6, 7, 13, 12, 18, 19, 20]	10.6	10
初始位置24	[20, 19, 18, 24, 18, 19, 20]	5.8	10
初始位置34	[20, 21, 27, 33, 34, 33, 32, 26, 20]	4.6	10
初始位置2	[20, 14, 8, 2, 8, 9, 15, 14, 20]	8.4	10
初始位置16	[20, 21, 22, 16, 22, 28, 27, 21, 20]	7.7	10
初始位置30	[20, 26, 32, 31, 30, 31, 32, 26, 20]	6.0	10
初始位置25	[20, 19, 25, 31, 32, 26, 20]	4.8	5
初始位置27	[20, 21, 27, 26, 20]	2.1	10
初始位置19	[20, 19, 20, 26, 20]	2.8	2
充电桩 A 返回	[20, 21, 27, 33, 34, 35]	2.2	0
充电桩 B 返回	[20, 19, 18, 12, 6, 0]	4.4	0

根据上表，可以看到有两条线路拿的书非常少（初始位置27、初始位置19），分别是2本和5本，而在这两条线路中需要取书的点恰好距离很近。所以我们将这两条线路合并到一起。根据计算和尝试，这些点经过并形成的最短路径应该是：[20, 26, 32, 31, 25, 19, 20]，比分开的两条路径要少2.8km。但通过人工分析只能得到对于特定路径的特定解法，对于应用到其他情形时不具有普适性，因此，将人脑的思考刨析为一种对于剩余数量的特殊算法对于提升模型适用性就极其重要。

我们认为当场上$\sum_i r_i \le 2C$时，机器人需要使用一套不同的算法寻找新的路线。因为在本问题中，当场上剩余量小于时20时，按照最优化路径的算法走，就很容易出现无法最大化利用机器人容载量的情况，而这种情况一旦出现，机器人就需要多出发一次，去将忽略掉的书本拾起。其后果就是会白白增加机器人行驶路径。因此，小于等于20时需要优先对于机器人承载效率进行优化，再对行驶路径进行优化。

我们先利用最短路算法寻找到剩余有图书的点中互相距离最高的两个点作为分类初始点。接着，分别找到与其相邻的含有剩余图书的点，并将分别将剩余量相加，得到两类点分别的剩余量，并“感染”为可以进行相邻点判定的点。以此类推，直至其中一类点的总剩余量达

到机器人最大承载量。在这个过程中，当含有剩余图书的相邻点中与其距离最小的一点被另一方“拿完”时，应寻找与其距离次低的点进行计算；而如果与其距离最小的相邻点中有剩余图书（因为对方背包容载量不够），可以将其中没有被对方占有的书的量划归为自己的类别中，直至所剩所有点和其中的书都被分配到两类点中。

以本题中最后剩余点的情况为例（如下图所示）。27、26（中的5本书）是第一类，后文中定义其为类 A；19、25、31、26（中的1本书）是第二类，后文中定义其为类 B。此时，机器人只需要出发2次，就可以将所有点的图书拿完，而不是对最终剩余量进行优化前的3次。此时，我们只需要分别对两类点进行路径规划即可。

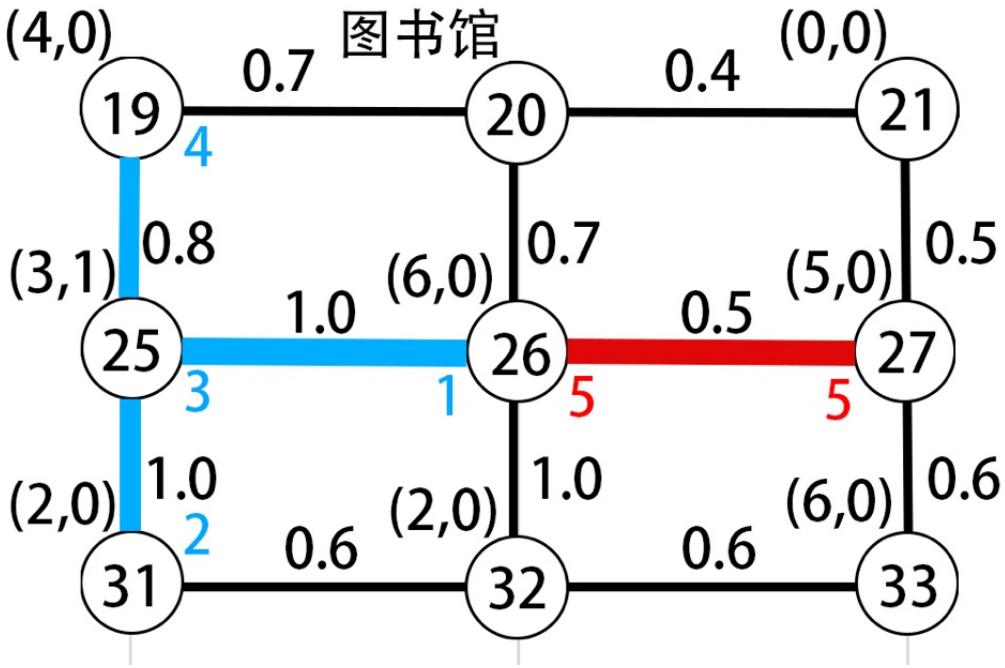


图 4. 最后20组书分类图

在分别对两类点的路径进行规划时，先判断在这类点中与原点（图书馆）距离最远的点并规划其到原点的最短路径作为分割线。计算节点相对于分割线的位置。如果位于分割线左侧，则记为负值并作为去程途径点；如果在分割线右侧，则记为正值并作为回程途径点；如果存在点在分割线上，则记为 0，并分别判断正值和负值的多少：如果去程途径点更多，则这个点归为回程途径点；如果回程途径点更多，则这个点归为去程途径点。下一步，使用最短路算法计算去程点相对于原点的最短路径并排序，作为去程途径点顺序，同理，计算回程途径点相对于距离原点最远点的最短距离并排序，作为回程途径点顺序。得到的顺序为完成此类点的行走顺序，例如 A 类点顺序为$[20, 19, 25, 31, 26, 20]$，B 类点顺序为$[20, 26, 27, 21, 20]$。

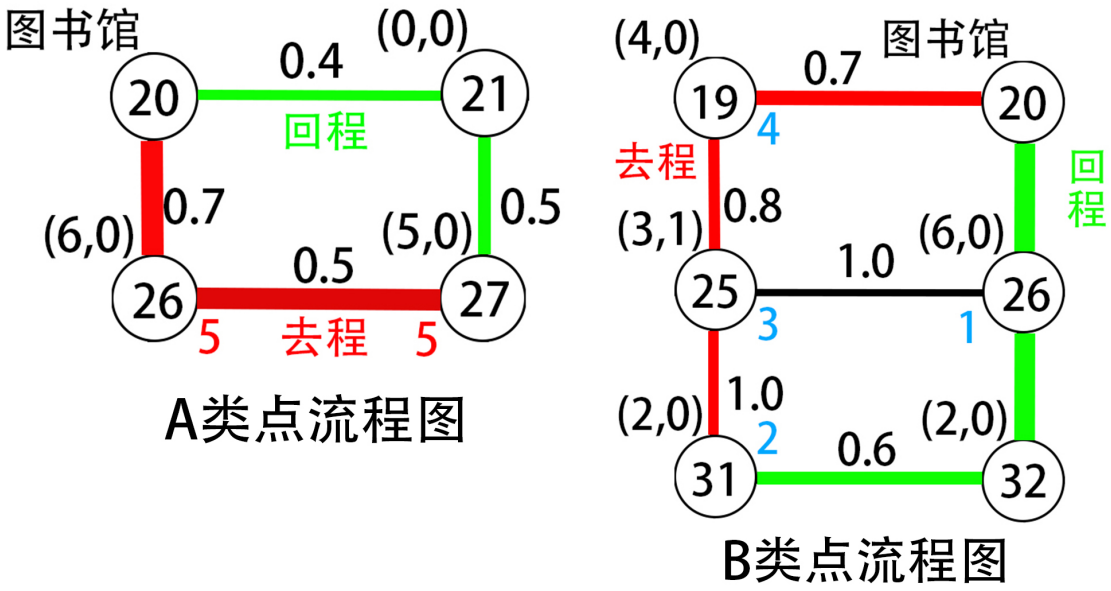


图 5. 两类点分别的路径规划结果

计算去程距离时，将原点作为起始点，距离原点最远点作为终点并依次使用最短路算法计算起始点到途径点的最短距离和先经过的途径点到下一个途径点间的距离，以及最后一个途径点到终点间的距离作为去程距离。同理，回城距离的计算将距离原点最远点作为起始点，原点作为终点，做相同的计算。去程距离与回城距离相加，为完成此类点需要行驶的最短距离。分别计算完成两类点的最短距离，得到优化结果。

6.5. 路程分配

当同时存在 AB 机器人时，由于两个机器人的速度不同，所分配的路线长度也不应相同。为了缩短时间，我们希望让 AB 两个机器人的线路按照各自的速度比例分配，并能同时回到充电桩。同时，由于不考虑机器人中途互换书和楼内取书时间，不同的线路顺序对总路程及时间不会产生影响，因此我们需要尽可能使分配后两机器人的路程关系为：

$$v_a : v_b = 8:10,\quad s_a : s_b \approx 8:10$$

经过尝试与分析，最终将所有路径分配成如下表所示：

表 3. 取书路径分配

机器人 A	机器人 B
充电桩 A 出发	充电桩 B 出发
初始位置25	初始位置27 + ~~初始位置19~~（已合并）
初始位置24	初始位置34
初始位置16	初始位置30

初始位置2	初始位置3
初始位置10	初始位置4
充电桩 A 返回	初始位置6
	充电桩 B 返回

经计算，最终完成所有还书需求的总耗时为5.09小时。

6.6. 灵敏度分析

机器人是否前往下一个点取决于下一个点的道路实用性及气口数量，但道路实用性只会与r_i是否为零有关，与r_i大小无关。因此，对于同一张地图来说，r_i的改变对实际路程的影响较小，灵敏度较低；但这也意味着对地图道路长度的修改会大幅度影响最终结果，因为道路长度与道路实用性之间有直接联系。

因为在本模型中机器人会尽可能将自己的背包填满并且不会跳过任何节点，机器人的承载量与速度对路线规划的影响则是近乎线性的。因此，在求解路线时，对承载量与速度的设定不会对方案的整体倾向产生影响，只会影响实际的路线闭合回路大小。

7. 借还书任务

7.1. 额外影响

为了同时考虑借书和还书的方法，我们需要对模型进行一些修改。与只考虑还书的模型相比，同时考虑借书时对决策的影响因素更多。在还书的同时考虑借书时，还需要考虑出发时需要携带的量。大多数时候，我们更自然的认为任何时候都要将机器人的容量使用到极限，但当借书量不满足远小于还书量时，在放下某点借阅书后，背包剩余容量可能会小于还书量，导致这个点的还书任务无法完成。在这种情况下，就需要一个点两次。此时，如果导致这样结果的原因是由于带了过多的书（包里仍有被借的书盈余），那带更多数量的书就是不值得的。因此，在考虑借书时，不能武断地认为从图书馆带出更多的书更好，而应该带出适量的书，以达到效率最大化。

当在一条路径中，存在某个节点因为后面节点借书量较大，无法让该节点的所有本该能装下的书全部装下的时候，我们可以考虑后面那个结点是否在其他路径中经过。在对每条路径所有节点借书量进行比较后，我们发现只有一个点的借书无法满足在经过该点取还书时拿取，这是因为该点只存在于机器人 B 从初始充电桩到图书馆的路径中。为了不影响其他路径和整体总路程，该点的借书任务会在最后从图书馆到充电桩的时候完成。据此，我们只将最后图书馆到充电站的路径进行重新规划，计算出最终的总路程是92.3km，时间约为5.13小时。

7.2. 模型异同对比

因为第一问中的模型完全是通过对影响因素的分析，是由对算法进行优化所得到。得到的模型具有更强的普适性，对于这个情形（地图，还书量布局）以外的情形，也具有适用性。而在第二问中，因为基本假设中我们假设了需要借书的数量远小于需要还书的数量，所以我们更优先地以还书量作为了选择最短路径的判断标准，并且只对无法同时完成借还书需求的路线进行优化。在这种情形下，需要进行优化的只有一个很小规模的路径，减轻了枚举负担。

8. 结论和进一步讨论

8.1. 整体评价

本文所提出的模型考虑了很多因素，其中包括楼宇的还书量、借书量、最短回程距离、每个节点的气口数量、以及与机器人相关的定量特性因素。我们希望通过考虑这些因素之间的优先级关系，让算法尽可能减少较远且实用性较低的道路的访问次数，从而降低多次重复访问的代价。值得一提的是，通过对比本文模型与使用遗传算法进行计算的行业成熟模型CVRP[4]，我们发现两者结果的总距离差异很小，但本文模型的运算时间相较 CVRP 缩短了一半以上（8秒对比25秒）。因此，我们认为在求解算力需求方面，本文所提出的模型相较 CVRP 模型更具竞争力，同时在结果性能上不输 CVRP 模型。

8.2. 优势和劣势

模型构建方面，本文使用结果稳定的决策树模型覆盖了大多数会影响总路程的因素，并将道路实用性也作为策略因素考虑，相比于 CVRP 的大量迭代在时间成本方面更有优势。不过，在假设方面，我们认为机器人每经过一个节点时会取完该点所有需要还的书，不会在一点只拿一定量的书。同时，我们发现在最后的几个路径中，存在没能拿到更多的书而增加总路径长度的情况。这些都是在构建模型时没能考虑到的因素，只能通过模型得出的数据再进行计算判断。在第二问中，我们运用了第一问所得出的结论进行分析，大幅度减少了运算时间。不过这是在借书量远小于还书量的情况下得到的结论，因此当借书量更大、机器人无法同时装载更多需要借阅和归还的书籍时，我们的模型无法通过研究仅还书情况得出的结论进行改善优化。

8.3. 未来研究方向

我们认为，前文中所提及的因素能否进一步在长期决策中影响总时间仍需深入考虑；对于借书还书任务，如何综合优化总路程距离及时间也应进行更多研究。本模型的构建是对传统 TSP 问题贪心思想的延伸，而在 TSP 问题中还存在其他思想以及算法可供寻找最优路径。

当得出总路径长度后，如何减少枚举计算的时间复杂度，并将其按一定比例分给 AB 两机器人对新的算法提出了更高的要求。

本文中所涉及到的优化方案是笔者通过对路径的分析得出的。但我们认为，单纯使用路径优化很难让优化后的方案达到最佳状态。因此，我们曾参考过经济学中自由竞争市场中的自然优化现象，即根据边际收入（MR）与边际成本（MC）来评估点的相对价值。但由于边际收入和边际成本的影响因素很多且内相关性很强，如何将影响因素考虑周全并列出合理的表达式成为此优化的一大难点。

同时，本文只对还书情形做出了普适性算法。而由于我们假设借书数量远小于还书数量，对于同时借与还的情形，我们选择进行单向耦合优化：即借还书会影响借书，但借书不会影响还书。未来通过建立借与还的双向耦合模型，希望能够解决任意借书数与还书数的更普遍的情形，将借还书路线的优化推向新的高度。

参考文献

[1] Reinelt, G. (2003). The traveling salesman: computational solutions for TSP applications (Vol. 840). Springer.

[2] Razali, N. M., & Geraghty, J. (2011, July). Genetic algorithm performance with different selection strategies in solving TSP. In Proceedings of the world congress on engineering (Vol. 2, No. 1, pp. 1-6). Hong Kong, China: International Association of Engineers.

[3] Kim, G., Ong, Y. S., Heng, C. K., Tan, P. S., & Zhang, N. A. (2015). City vehicle routing problem (city VRP): A review. IEEE Transactions on Intelligent Transportation Systems, 16(4), 1654-1666.

[4] Eksioglu, B., Vural, A. V., & Reisman, A. (2009). The vehicle routing problem: A taxonomic review. Computers & Industrial Engineering, 57(4), 1472-1483.

[5] Toth, P., & Vigo, D. (Eds.). (2002). The vehicle routing problem. Society for Industrial and Applied Mathematics.

[6] Baker, B. M., & Ayechew, M. (2003). A genetic algorithm for the vehicle routing problem. Computers & Operations Research, 30(5), 787-800.

[7] Mazzeo, S., & Loiseau, I. (2004). An ant colony algorithm for the capacitated vehicle routing. Electronic Notes in Discrete Mathematics, 18, 181-186.

[8] Tavakkoli-Moghaddam, R., Safaei, N., Kah, M. M. O., & Rabbani, M. (2007). A new capacitated vehicle routing problem with split service for minimizing fleet cost by simulated annealing. Journal of the Franklin Institute, 344(5), 406-425.

附录

/requirements.txt （库依赖）

```
networkx
matplotlib
```

/defines.py （常量定义）

```
SPEED_A = 8
SPEED_B = 10
BP_VOL = 10

WIDTH = 6
HEIGHT = 6
P_NUM = WIDTH * HEIGHT

CHARGER_A = 35
CHARGER_B = 0
LIBRARY = 20
CROSS1 = 14
CROSS2 = 21
```

/graph.py （图模块 + 主程序）

```
from defines import *
import networkx as nx
import matplotlib.pyplot as plt
import copy
import sys

sys.setrecursionlimit(1000)


class Graph:
    def __init__(self, filepath='./data/graph.txt'):
        self.graph = []
        self.accu = {}
        self.accu_max = None
        self.accu_min = None
        for i in range(P_NUM):
            self.graph.append({"r": None, "b": None, "e": []})
```

```
        with open(filepath, "r") as f:
            file_lines = f.readlines()
            cur_p = -1
            for line in file_lines:
                line = line.replace("\n", "")
                if cur_p == -1:
                    cur_p = int(line)
                    continue
                if self.graph[cur_p]["r"] is None:
                    r, b = line.split(" ")
                    r = int(r)
                    b = int(b)
                    self.graph[cur_p]["r"] = r
                    self.graph[cur_p]["b"] = b
                    continue
                if line == "end":
                    cur_p = -1
                    continue
                v, dist = line.split(" ")
                v = int(v)
                dist = float(dist)
                self.graph[cur_p]["e"].append((v, dist))
                self.accu[(cur_p, v)] = None

        self._g = nx.MultiDiGraph()
        for i in range(P_NUM):
            self._g.add_node(i)
        for i in range(P_NUM):
            vertex = self.get_vertex(i)
            edges = vertex["e"]
            for edge in edges:
                self._g.add_edge(i, edge[0], length=edge[1])

        self.paths = None
        self.refresh_all()

    def refresh_all(self):
        self.refresh_paths()
        self.refresh_accu()
        self.refresh_accu_extrema()

    def refresh_paths(self):
        self.paths = nx.shortest_path(
            G=self._g,
```

```
            source=None,
            target=None,
            weight="length",
            method="dijkstra"
        )

    def get_path_len(self, path):
        return nx.path_weight(self._g, path, "length")

    def get_path(self, source, target):
        return self.paths[source][target], self.get_path_len(self.paths[source][target])

    def get_vertex(self, i):
        return self.graph[i]

    def plot_graph(self):
        nx.draw(
            G=self._g,
            pos=nx.spring_layout(self._g)
        )
        plt.show()

    @staticmethod
    def id2pos(id):
        return id // WIDTH, id % WIDTH

    def refresh_accu(self):
        for key in self.accu.keys():
            self.accu[key] = 0
        for i in range(P_NUM):
            if self.get_vertex(i)["r"] == 0:
                continue
            path, path_len = self.get_path(LIBRARY, i)
            for j in range(len(path) - 1):
                u = path[j]
                v = path[j + 1]
                self.accu[(u, v)] += 1
                self.accu[(v, u)] += 1

    def refresh_accu_extrema(self):
        self.accu_max = 0
        for key in self.accu.keys():
            self.accu_max = max(self.accu_max, self.accu[key])
```

```
105.            self.accu_min = self.accu_max
106.            for key in self.accu.keys():
107.                if self.accu[key] != 0:
108.                    self.accu_min = min(self.accu_min, self.accu[key])
109.
110.        def plot_accu(self, linewidth=5):
111.            ax = plt.gca()
112.            ax.invert_yaxis()
113.            for key in self.accu.keys():
114.                y1, x1 = Graph.id2pos(key[0])
115.                y2, x2 = Graph.id2pos(key[1])
116.                plt.plot([x1, x2], [y1, y2], marker="o", color="black", linewidth
    =linewidth,
117.                         alpha=self.accu[key] / self.accu_max / 2)
118.            plt.show()
119.
120.        def get_edge_len(self, i, j):
121.            return self.get_path_len([i, j])
122.
123.        def remove_edge(self, i, j, refresh=False):
124.            self._g.remove_edge(i, j)
125.            self._g.remove_edge(j, i)
126.
127.            self.path = None
128.
129.            for k in range(self.graph[i]["e"]):
130.                if self.graph[i]["e"][k][0] == j:
131.                    self.graph[i]["e"].pop(k)
132.            for k in range(self.graph[j]["e"]):
133.                if self.graph[j]["e"][k][0] == i:
134.                    self.graph[j]["e"].pop(k)
135.
136.            self.accu.pop((i, j))
137.            self.accu.pop((j, i))
138.
139.            if refresh:
140.                self.refresh_all()
141.
142.        def set_r(self, i, r, refresh=False):
143.            self.graph[i]["r"] = r
144.            if refresh:
145.                self.refresh_accu()
146.
147.        def is_finish_r(self):
```

```
148.         for i in range(P_NUM):
149.             if self.get_vertex(i)["r"] > 0:
150.                 return False
151.         return True
152.
153.     def get_accu(self, i, j):
154.         return self.accu[(i, j)]
155.
156.     def get_utils(self, i, remain, strict=1):
157.         graph = self.copy()
158.         enough = False
159.         if graph.get_vertex(i)["r"] <= remain:
160.             enough = True
161.             remain -= graph.get_vertex(i)["r"]
162.             graph.set_r(i, 0, refresh=True)
163.         else:
164.             remain = 0
165.         vertex = graph.get_vertex(i)
166.         edges = vertex["e"]
167.         cnt = 0
168.         utils_list = []
169.         for edge in edges:
170.             v = edge[0]
171.             util = Graph.calc_util(graph, v)
172.             if strict is not None:
173.                 if util == strict:
174.                     cnt += 1
175.                     utils_list.append(v)
176.             else:
177.                 if util > 0:
178.                     cnt += 1
179.                     utils_list.append((v, util))
180.         return cnt, utils_list, graph, enough, remain
181.
182.     def get_future_utils(self, i, remain, prev):
183.         remain_bak = remain
184.         cnt, utils_list, graph, enough, remain = self.get_utils(i, remain)
185.         if remain == 0:
186.             return [i], True, cnt, enough
187.         if cnt > 0:
188.             v_utils = []
189.             v_utils_strict1 = []
190.             for v in utils_list:
191.                 if v == prev:
```

```
                        continue
                    v_path, v_strict1, v_cnt, v_enough = graph.get_future_utils(v
, remain, i)
                    v_utils.append((v_path, v_cnt, v_enough))
                    if v_strict1:
                        v_utils_strict1.append((v_path, v_cnt, v_enough))
                if False:
                    """
                    max_v_cnt = -1
                    max_v_path = None
                    for v_util in v_utils_strict1:
                        (v_path, v_cnt, v_enough) = v_util
                        if max_v_cnt < v_cnt:
                            max_v_cnt = v_cnt
                            max_v_path = v_path
                    max_v_path.insert(0, i)
                    return max_v_path, True, cnt, enough
                    """
                    enough_list = []
                    not_enough_list = []
                    for v_util in v_utils_strict1:
                        (v_path, v_cnt, v_enough) = v_util
                        if v_enough:
                            enough_list.append(v_util)
                        else:
                            not_enough_list.append(v_util)
                    if len(enough_list) > 0:
                        max_v_dist = -1
                        max_v_path = None
                        for v_util in enough_list:
                            (v_path, v_cnt, v_enough) = v_util
                            dist = self.get_path(v_path[0], LIBRARY)[1]
                            if dist > max_v_dist:
                                max_v_dist = dist
                                max_v_path = v_path
                        max_v_path.insert(0, i)
                        return max_v_path, True, cnt, enough
                    else:
                        min_v_dist = None
                        min_v_path = None
                        for v_util in not_enough_list:
                            (v_path, v_cnt, v_enough) = v_util
                            dist = self.get_path(v_path[0], LIBRARY)[1]
                            if min_v_dist is None or dist < min_v_dist:
```

```
                            min_v_dist = dist
                            min_v_path = v_path
                    min_v_path.insert(0, i)
                    return min_v_path, True, cnt, enough
            else:
                """
                max_dist = -1
                max_v_path = []
                for v_util in v_utils:
                    (v_path, v_cnt) = v_util
                    v = v_path[0]
                    dist = self.get_path(v, LIBRARY)[1]
                    if max_dist < dist:
                        max_dist = dist
                        max_v_path = v_path
                max_v_path.insert(0, i)
                return max_v_path, False, cnt, enough
                """
                enough_list = []
                not_enough_list = []
                for v_util in v_utils:
                    (v_path, v_cnt, v_enough) = v_util
                    if v_enough:
                        enough_list.append(v_util)
                    else:
                        not_enough_list.append(v_util)
                if len(enough_list) > 0:
                    max_v_dist = -1
                    max_v_path = None
                    for v_util in enough_list:
                        (v_path, v_cnt, v_enough) = v_util
                        dist = self.get_path(v_path[0], LIBRARY)[1]
                        if dist > max_v_dist:
                            max_v_dist = dist
                            max_v_path = v_path
                    max_v_path.insert(0, i)
                    return max_v_path, True, cnt, enough
                else:
                    min_v_dist = None
                    min_v_path = None
                    for v_util in not_enough_list:
                        (v_path, v_cnt, v_enough) = v_util
                        dist = self.get_path(v_path[0], LIBRARY)[1]
                        if min_v_dist is None or dist < min_v_dist:
```

```
                                min_v_dist = dist
                                min_v_path = v_path
                        min_v_path.insert(0, i)
                        return min_v_path, True, cnt, enough
            else:
                remain = remain_bak
                cnt, utils_list, graph, enough, remain = self.get_utils(i, remain
    , strict=None)
                v_utils = []
                all_not_enough = False
                for (v, util) in utils_list:
                    if v == prev:
                        continue
                    v_path, v_strict1, v_cnt, v_enough = graph.get_future_utils(v
    , remain, i)
                    v_utils.append((v_path, v_cnt, v_enough))
                    all_not_enough = all_not_enough or v_enough
                all_not_enough = not all_not_enough
                """
                if all_not_enough:
                    min_dist = None
                    min_v_path = []
                    for v_util in v_utils:
                        v_path, v_cnt, v_enough = v_util
                        v = v_path[0]
                        dist = self.get_path(v, LIBRARY)[1] + self.get_edge_len(i
    , v)
                        if min_dist is None or min_dist > dist:
                            min_dist = dist
                            min_v_path = v_path
                    min_v_path.insert(0, i)
                    return min_v_path, False, 0, enough
                else:
                    v_path = []
                    if len(v_utils) > 0:
                        v_util = v_utils[0]
                        v_path, v_cnt, v_enough = v_util
                    v_path.insert(0, i)
                    return v_path, False, 0, enough
                """
                enough_list = []
                not_enough_list = []
                for v_util in v_utils:
                    (v_path, v_cnt, v_enough) = v_util
```

```
                if v_enough:
                    enough_list.append(v_util)
                else:
                    not_enough_list.append(v_util)
            if len(enough_list) > 0:
                max_v_dist = -1
                max_v_path = None
                for v_util in enough_list:
                    (v_path, v_cnt, v_enough) = v_util
                    dist = self.get_path(v_path[0], LIBRARY)[1]
                    if dist > max_v_dist:
                        max_v_dist = dist
                        max_v_path = v_path
                max_v_path.insert(0, i)
                return max_v_path, True, cnt, enough
            else:
                min_v_dist = None
                min_v_path = []
                for v_util in not_enough_list:
                    (v_path, v_cnt, v_enough) = v_util
                    dist = self.get_path(v_path[0], LIBRARY)[1]
                    if min_v_dist is None or dist < min_v_dist:
                        min_v_dist = dist
                        min_v_path = v_path
                min_v_path.insert(0, i)
                return min_v_path, True, cnt, enough

    @staticmethod
    def calc_util(graph, i):
        vertex = graph.get_vertex(i)
        edges = vertex["e"]
        util = 0
        for edge in edges:
            v = edge[0]
            util += graph.get_accu(i, v)
        return util

    def copy(self):
        return copy.deepcopy(self)

    def exec_r(self, i, charger=False):
        path = self.get_future_utils(i, BP_VOL, None)[0]
        total_dist = self.get_path_len(path)
        if not charger:
```

```
364.                 total_dist += self.get_path(LIBRARY, i)[1]
365.             total_dist += self.get_path(path[len(path) - 1], LIBRARY)[1]
366.             remain = BP_VOL
367.             for v in path:
368.                 vertex = self.get_vertex(v)
369.                 if remain >= vertex["r"]:
370.                     remain -= vertex["r"]
371.                     self.set_r(v, 0)
372.                 else:
373.                     self.set_r(v, vertex["r"] - remain)
374.                     remain = 0
375.             taken = BP_VOL - remain
376.             self.refresh_accu()
377.             if not charger:
378.                 path = self.get_path(LIBRARY, i)[0][:-1] + path
379.             path += self.get_path(path[len(path) - 1], LIBRARY)[0][1:]
380.             return total_dist, taken, path
381.
382.         def pick_start(self):
383.             max_util = -1
384.             max_util_list = []
385.             for i in range(P_NUM):
386.                 if self.get_vertex(i)["r"] == 0:
387.                     continue
388.                 if self.calc_util(self, i) > 1:
389.                     continue
390.                 util = self.get_utils(i, BP_VOL)[0]
391.                 dist = self.get_path(i, LIBRARY)[1]
392.                 if max_util < util:
393.                     max_util = util
394.                     max_util_list = []
395.                 if max_util == util:
396.                     max_util_list.append((i, dist))
397.             max_dist = 0
398.             max_v = None
399.             for v in max_util_list:
400.                 if max_dist < v[1]:
401.                     max_dist = v[1]
402.                     max_v = v[0]
403.             return max_v
404.
405.         def run_all_r(self):
406.             self.plot_accu()
```

```
407.          dist_charger_A, taken_charger_A, path_charger_A = self.exec_r(CHARGER
   _A, charger=True)
408.          dist_charger_B, taken_charger_B, path_charger_B = self.exec_r(CHARGER
   _B, charger=True)
409.          path_charger_A_back, dist_charger_A_back = self.get_path(LIBRARY, CHA
   RGER_A)
410.          path_charger_B_back, dist_charger_B_back = self.get_path(LIBRARY, CHA
   RGER_B)
411.          charger_dist = dist_charger_A + dist_charger_B + dist_charger_A_back
   + dist_charger_B_back
412.          total_dist = charger_dist
413.          paths = [
414.              ("charger_A", (path_charger_A, dist_charger_A, taken_charger_A)),
415.              ("charger_B", (path_charger_B, dist_charger_B, taken_charger_B)),
416.              ("charger_A_back", (path_charger_A_back, dist_charger_A_back, 0))
   ,
417.              ("charger_B_back", (path_charger_B_back, dist_charger_B_back, 0))
418.          ]
419.          start_point_list = []
420.          self.plot_accu()
421.          run_i = 0
422.          while not self.is_finish_r():
423.              start_point = self.pick_start()
424.              start_point_list.append(start_point)
425.              dist, taken, path = self.exec_r(start_point)
426.              total_dist += dist
427.              paths.append((start_point, (path, dist, taken)))
428.
429.              run_i += 1
430.              print("Iteration %d complete." % run_i)
431.              self.plot_accu()
432.          self.plot_accu()
433.
434.          return total_dist, charger_dist, (total_dist - charger_dist), start_p
   oint_list, paths
435.
436.
437.  if __name__ == "__main__":
438.      G = Graph()
439.      total_dist, charger_dist, lib2lib_dist, start_point_seq, paths = G.run_al
   l_r()
440.      print("-----------------------")
441.      print("Total distance: %.1f" % total_dist)
442.      print("Charger distance: %.1f" % charger_dist)
```

```
443.      print("Lib2lib distance: %.1f" % lib2lib_dist)
444.      print("-----------------------")
445.      print("Lib2lib starting points:", start_point_seq)
446.      print("-----------------------")
447.      for i in range(len(paths)):
448.          print("ROUTE %d" % (i + 1))
449.          print("Starting from %s" % paths[i][0])
450.          print("Path:", paths[i][1][0])
451.          print("Distance: %.1f" % paths[i][1][1])
452.          print("Taken: %d" % paths[i][1][2])
453.          print("-----------------------")
```

/data/graph.txt （图数据）

见支持文件